B1 window GRU-only

Phase 4 Court Judgment Prediction | source-audited documentation

Problem statement and data

Phase 4 Court Judgment Prediction predicts binary outcome 0/1 from legal case text. The preserved official inputs are Train/train.json, Train/dev.json and Test/test.json. Test labels are private. Original run records report 994 official dev labels and 1,517 test IDs; macro-F1 is the central metric.

Architecture and mechanics

B1 6-encoder/6-decoder original Phase 1 backbone; 512 tokens per chunk, 16 ordered chunks per case, overlap 100, tail sampling. Document BiGRU without document Transformer. Token representation uses a learned mixture of mean, attention and last content token; position enters before document aggregation. The final learned attention pools document states for two-class logits. Data flow: Legal case -> B1 6+6 backbone -> 512×16 windows -> Token evidence mix -> Document BiGRU without document Transformer -> Attention pool -> Binary ZIP.

Training design and checkpoint selection

Staged B1 fine-tuning; document head mode gru_only. Original B1 study notebook lineage code/v1.ipynb through code/v7.ipynb; latest source snapshot code/source/b1_study/. Saved configs and terminal/run evidence are in evidence/.. Original B1 study notebook lineage code/v1.ipynb through code/v7.ipynb; latest source snapshot code/source/b1_study/. Saved configs and terminal/run evidence are in evidence/.

Exact configuration, training progress and compute

Head mode gru_only; 6 encoder blocks, no FFN expansion in this scored configuration; window policy tail, 512 tokens per window, 16 windows, overlap 100. Backbone learning rate 1e-05, head learning rate 0.0003, accumulation 8, warmup updates 100, target maximum epochs 4. Latest recorded segment 3 in this package; selected step 5000, optimizer updates 7465, completed full epochs 1, seen training cases 59716. Selected checkpoint SHA-256 d01f8978e9ac2d13eee5d202fa75df285c27915fecdbbc0909f72703466d8c80; peak GPU bytes 1973151232. The preserved lineage JSON traces the resumed optimizer/checkpoint state across segments; the ZIP is produced from a selected checkpoint, not an invented continuation.

Measured results

Version 7 run state paused; 7465 optimizer updates and 1 complete epochs, selected step 5000. Calibrated official 994-case dev macro-F1 0.619677 at threshold 0.447596; default 0.5 dev macro-F1 0.649809. Selected checkpoint SHA-256 d01f8978e9ac2d13eee5d202fa75df285c27915fecdbbc0909f72703466d8c80; verified 1,517-ID ZIP SHA-256 796c249f600dbd3140102ce45fd797b397ccc2277bcf1b11693167fc9f2a5455. No finished Codabench score for this exact export is established by the supplied screenshot.

Evaluation mechanics

Official labeled dev cases 994; balanced class supports [497, 497]; selected-threshold confusion matrix [[232, 265], [103, 394]]; calibrated threshold 0.447596 and macro-F1 0.619677; default 0.5 macro-F1 0.649809. These are two evaluations of the same selected weights, with different binary decision thresholds. The 1,517-ID test prediction ZIP is a separate label-free artifact.

Prediction contract and verification

The original prediction ZIP contains one root-level prediction.json with 1,517 unique official test IDs and integer labels restricted to 0 and 1. This package checks the ZIP CRC and SHA-256 against the archived run/verification record. Full model checkpoint tensors remain outside GitHub; their downloaded backup receipts and hashes live in the local Phase 4 study archives.

Reproducibility and limits

A budget-paused run is not converged; the target was four epochs with checkpoint selection and train-only calibration. The v7 source also defines gated depth expansion and an upper FFN option, but the recorded configs for these exports keep encoder_depth=6 and expand_ffn=false; those unlaunched options have no reported score. Original checkpoint bytes were downloaded and verified in the local archive, but are omitted from GitHub. Reports and companion notebook were created after training; original scores come from copied artifacts. Model weights and course data are preserved locally, not uploaded to GitHub.